SO - Input / Output

1 I/O Hardware

Quali sono i Dispositivi di I/O?

I dispositivi di I/O si possono suddividere in:

- **Dispositivi a blocchi:** Archiviano informazioni in blocchi di dimensioni fisse. Ogni blocco è indipendente dagli altri, e i trasferimenti sono in unità di uno o più blocchi interi.
  ESEMPI: dischi fissi magnetici, SSD, ecc.
- **Dispositivi a caratteri:** Rilasciano o accettano un flusso di caratteri senza alcuna struttura a blocchi.
  ESEMPI: stampanti, mouse, tastiere, ecc.

Come sono composti i Dispositivi di I/O?

Sono composti da:

- una parte elettronica chiamata controller del dispositivo, è un chip o una scheda a circuiti stampati
- una parte meccanica, ovvero il dispositivo stesso.

Le interfacce fra il controller e il dispositivo possono essere standardizzate come ANSI, IEEE, ISO o de facto, come SATA, SCSI, USB o Thunderbolt.

Il compito del controller è convertire il flusso seriale di bit in blocchi di byte, correggere errori e trasferire in memoria centrale.
Senza il controller, il programmatore dovrebbe gestire dettagli complessi, come la modulazione di ciascun pixel.
Il controller viene inizializzato dal sistema operativo con parametri essenziali, poi viene delegato alla gestione dei dettagli complessi.
Il controller dispone di alcuni registri per gestire le comunicazioni con la CPU. Leggendo da questi registri il Sistema Operativo riesce a capire quale sia lo stato del dispositivo.

Porta Parallela vs Porta USB

- **Porta Parallela:** Interfaccia di comunicazione tra computer e dispositivi. Dotata di 25 o 36 pin.
  - Pin 1-8: trasferimento di dati
  - Pin 9-16: controlli e status
  - Pin 17-25: masse e alimentazione
- **Porta USB:** Interfaccia standardizzata di comunicazione tra computer e dispositivi Utilizzata per fornire alimentazione elettrica oltre che dati. Dotata da 4 pin:
  - Pin 1: VCC alimentazione 5V
  - Pin 2: D- Dati negativi
  - Pin 3: D+ Dati positivi
  - Pin 4: GND Terra

Come comunica la CPU con i dispositivi di I/O?

Esistono 2 metodi:

- *Port Mapped I/O* - assegna a ogni registro di controllo un numero di porta di I/O associato.
- *Memory Mapped I/O* - assegna a ogni registro di controllo un indirizzo di memoria univoco.

Direct Memory Access (DMA)

Il DMA permette alla CPU di accedere direttamente alla Memoria. Più nello specifico, possiamo confrontare il trasferiemnto di dati:

- SENZA DMA:La CPU chiede un dato al disco. Successivamente aspetta che il disco sia pronto. Appena il dato è nel buffer del controller, la CPU deve leggerlo e poi scriverlo nella RAM.
- CON DMA: La CPU prepara il trasferimento "istruendo" il controller DMA. Il controller DMA prende il comando e richiede formalmente al controller del disco di iniziare l'estrazione dei dati. Il controller del disco trasferisce i dati direttamente verso la memoria RAM. Per ogni blocco di dati trasferito, il controller del disco invia una conferma al DMA.

Per fare ciò, il controller DMA deve appropriarsi delle "corsie preferenziali" del computer, ovvero il BUS di sistema. Esistono tre modalità principali con cui il DMA interagisce con il BUS per spostare i dati:

- *Cycle Stealing:* Richiede il BUS alla CPU solo per il tempo necessario a trasferire una singola parola Qui, il DMA trasferisce una parola per volta, "rubando" cicli alla CPU nel caso in cui anche alla CPU servisse il BUS.
- *Modalità Burst:* Una volta ottenuto il controllo del BUS, lo occupa esclusivamente finché non ha terminato il trasferimento dell'intero blocco di dati richiesto. Qui, il DMA ottiene il controllo completo del bus, eseguendo trasferimenti multipli in una volta.
- *Fly by Mode:* Il DMA configura il BUS in modo che il controller del disco e la memoria RAM comunichino direttamente e simultaneamente. Trasferisce dati direttamente alla memoria principale senza intermediari.

Gli Interrupt

L'interrupt è un segnale asincrono inviato alla CPU da hardware o software per segnalare un evento urgente che richiede attenzione immediata, interrompendo temporaneamente il flusso di esecuzione del programma corrente.
Possono essere causati da diverse cause:

- Trap: azione deliberata dal codice, come ad esempio una chiamata di sistema
- Eccezione o Fault: errori di segmentazione o divisioni per zero
- Interrupt Hadware: segnali inviati da dispositivi come stampanti

Un dispositivo di I/O invia un segnale di Interrupt su una linea del bus quando ha terminato il suo compito.
L'interrupt viene ricevuto e gestito dal controller degli interrupt.
Se non ci sono altri interrupt il controller gestisce subito l'interrupt.
In caso di interrupt simultanei il dispositivo viene ignorato temporaneamente e continua a mandare segnali finchè non viene gestito.

Gestione degli Interrupt:

- **Segnalazione dell'Interrupt alla CPU:** Il controller assegna un numero alla linea del bus per specificare il dispositivo che richiede attenzione e invia un segnale di interruzione alla CPU.
- **Interruzione e Gestione da Parte della CPU:** La CPU interrompe il suo attuale compito. Utilizza il numero sulla linea degli indirizzi e lo utilizza come indice nella tabella vettore degli interrupt
- **Il Vettore degli Interrupt:**
- **Conferma e Gestione degli Interrupt:** La procedura di servizio conferma l'interrupt scrivendo su una porta del controller degli interrupt.
- **Salvataggio dello Stato:**

Gli interrupt sono di 2 tipologie:

- **Interrupt Precisi:** Situazione in cui il sistema pu'o determinare con esattezza quali istruzioni sono state completate al momento dell'interrupt e quali no.
- **Interrupt Imprecisi:** Situazione in cui diverse istruzioni vicine al program counter si trovano in vari stati di completamento al momento dell'interrupt.

2 I/O Software

Obiettivi del Software di I/O

- **Indipendenza dal Dispositivo**: Possibilità di accedere a diversi hardware senza dover riscrivere il software per ogni modello specifico.
- **Denominazione Uniforme**: Utilizzo di stringhe o numeri (es. nomi di file) per identificare i dispositivi in modo coerente.
- **Gestione degli Errori**: Gli errori devono essere risolti il più vicino possibile all'hardware (es. dal driver o dal controller).
- **Sincronia vs Asincronia**: Sebbene l'hardware lavori in modo asincrono, il software deve permettere ai programmi di operare in modalità bloccante (sincrona) per semplicità.
- **Buffering**: Utilizzo di memorie temporanee per gestire i dati che non possono essere inviati direttamente alla destinazione finale.

Tipologie di Gestione del Software

Esistono tre metodi principali per gestire il trasferimento dei dati:

2.0.1 I/O Programmato

La CPU si occupa interamente del trasferimento. Invia un dato e rimane in un ciclo di *polling* (o busy waiting), controllando continuamente lo stato del dispositivo finché non è pronto per il dato successivo. È inefficiente perché occupa la CPU a tempo pieno.

2.1 I/O Guidato dagli Interrupt

Per evitare lo spreco di cicli CPU, il sistema invia un dato e poi esegue lo *scheduling* di un altro processo. Quando il dispositivo è pronto, genera un **interrupt** per avvisare la CPU, che riprende brevemente il controllo per inviare il carattere successivo.

2.2 I/O con DMA (Direct Memory Access)

Il lavoro è delegato a un controller speciale (DMA). La CPU istruisce il DMA sul buffer da trasferire e si dedica ad altro. Viene generato un solo interrupt alla fine dell'intero trasferimento del buffer, riducendo drasticamente il carico sulla CPU.

Struttura a Livelli del Software di I/O

Il software è organizzato in quattro livelli gerarchici:

1. **Gestori degli Interrupt**: Il livello più basso. Si occupa di salvare lo stato della CPU, eseguire la procedura di servizio dell'interrupt e notificare il completamento al controller degli interrupt.

2. **Driver dei Dispositivi**: Contengono il codice specifico fornito dal produttore. Comunicano direttamente con i registri del controller hardware e sono spesso caricati dinamicamente nel kernel.

3. **Software del SO Indipendente dal Dispositivo**: Funge da ponte. Standardizza l'interfaccia per i driver, gestisce il buffering, l'allocazione dei dispositivi dedicati e l'uniformità della dimensione dei blocchi.

4. **Software a Livello Utente**: Include le librerie di sistema (come *stdio.h* in C) che forniscono funzioni semplici come `read()` o `printf()`. Qui si colloca anche lo **Spooling**, una tecnica che usa directory e processi *daemon* per gestire l'accesso sequenziale a dispositivi dedicati (es. stampanti).